

Frontend Technical Documentation

Next.js customer/chef web architecture and flows.



Summary

This document explains the subject in straightforward professional English, then adds the engineering detail needed to build, operate, troubleshoot or review it. Claims are based on the repository evidence snapshot; missing source details are called out instead of guessed.

Document details

Edition 2.0 | Evidence date: 14 August 2026 | Repository:
rmorampudi09-arch/Craves-Build-platform | Product evidence snapshot: main @
3225f7fa531a6b07185fb5e034f7930f8bd8b571

Summary and how to use this document

Next.js customer/chef web architecture and flows. The purpose is to make the system understandable to a new team member while still giving engineers enough detail to trace ownership, data flow, deployment impact and failure handling.

Current implementation

Direct code, README, migration, pipeline or commit evidence exists on the reviewed main snapshot.

Foundation / partial

Useful groundwork exists, but the repository does not prove a complete production runtime for every part.

Legacy / reference

Older implementation is retained for history or compatibility and is not treated as the preferred runtime.

Needs source confirmation

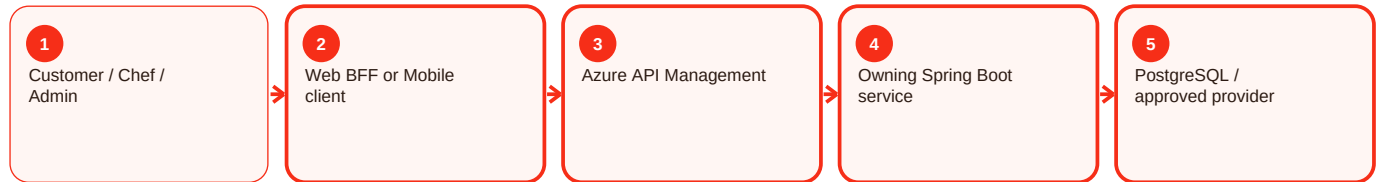
A referenced artifact or exact value could not be verified and is therefore not invented.

Reading order

- Start with the summary to understand the responsibility in normal language.
- Use process diagrams to follow requests across client, APIM, services, data and providers.
- Use the troubleshooting sections to diagnose by evidence rather than by retrying blindly.
- Use deployment and validation sections before or after changing a component.
- Use evidence status to separate proven behavior from gaps and future work.

How the request moves through Craves

The exact components depend on the feature, but the normal pattern is consistent. The user interface starts the request, the gateway and owning service enforce trust, the service writes authoritative state, and provider integrations remain behind controlled boundaries.



Key rule

A screen can hide or show an action for usability, but that is not security. APIM and the owning backend still validate the caller, role, ownership and current business state. Likewise, provider success is not accepted as Craves state until the backend verifies and records the result.

Next.js app router - Overview and responsibility

Current implementation

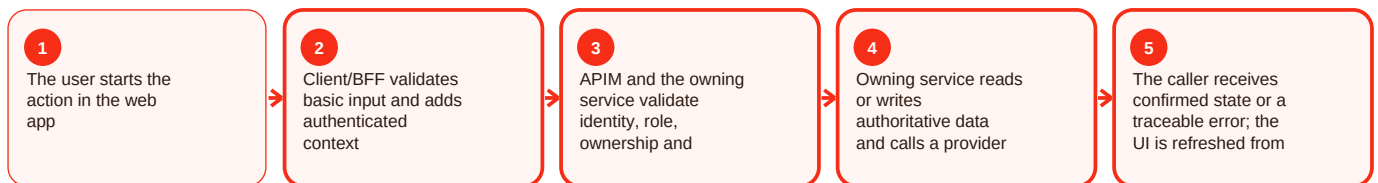
Summary

Next.js app router is part of the frontend area of Craves. The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state. For Next.js App Router, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Generic upstream error: BFF rejected malformed/unexpected backend response. Resolution: Use request ID and compare the current response contract.

Amount differs: Client state is stale. Resolution: Render backend-authoritative cart/quote state.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Next.js app router - Functional behavior and data

Current implementation

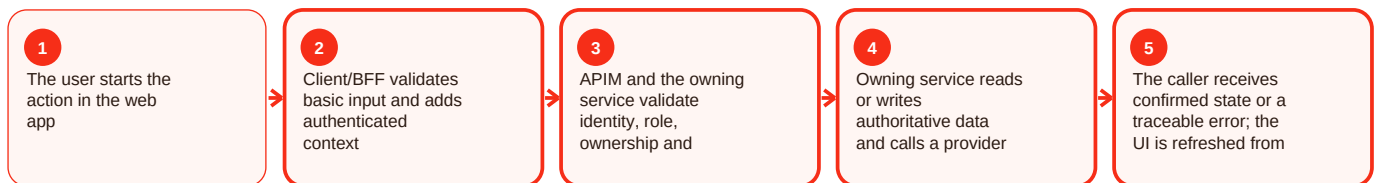
Summary

Next.js app router is part of the frontend area of Craves. The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state. For Next.js App Router, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Generic upstream error: BFF rejected malformed/unexpected backend response. Resolution: Use request ID and compare the current response contract.

Amount differs: Client state is stale. Resolution: Render backend-authoritative cart/quote state.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Next.js app router - Operations, errors and deployment

Current implementation

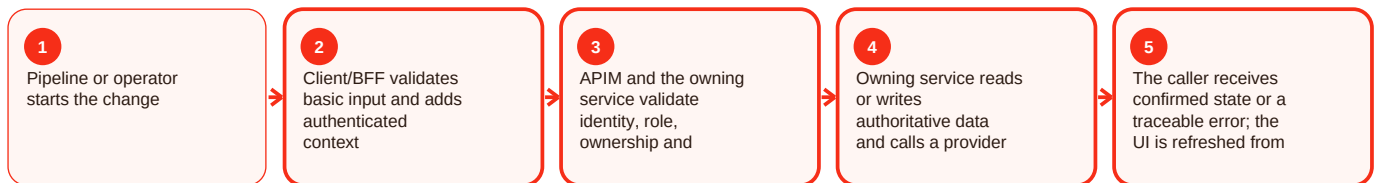
Summary

Next.js app router is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For Next.js App Router, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Same-origin bff - Overview and responsibility

Current implementation

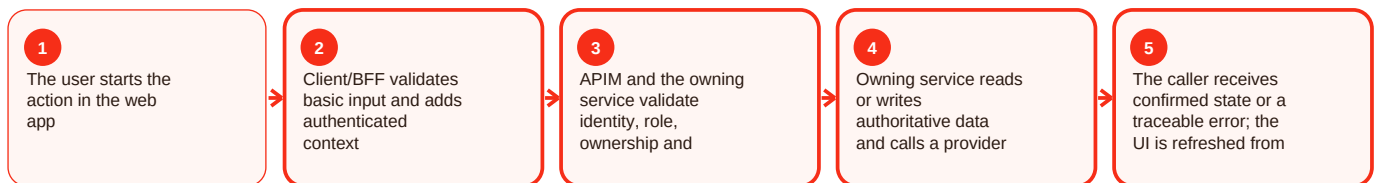
Summary

Same-origin bff is part of the frontend area of Craves. The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state. For same-origin BFF, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Generic upstream error: BFF rejected malformed/unexpected backend response. Resolution: Use request ID and compare the current response contract.

Amount differs: Client state is stale. Resolution: Render backend-authoritative cart/quote state.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Same-origin bff - Functional behavior and data

Current implementation

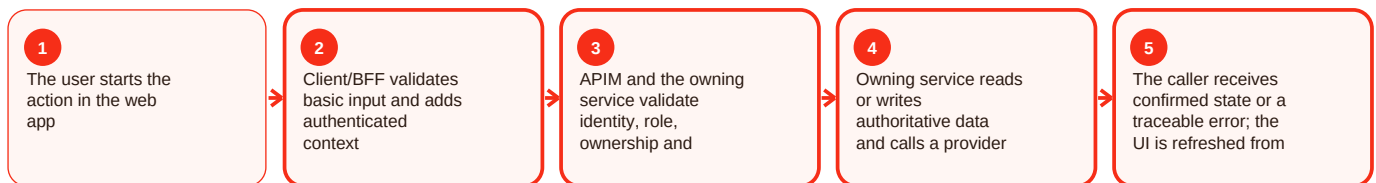
Summary

Same-origin bff is part of the frontend area of Craves. The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state. For same-origin BFF, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Generic upstream error: BFF rejected malformed/unexpected backend response. Resolution: Use request ID and compare the current response contract.

Amount differs: Client state is stale. Resolution: Render backend-authoritative cart/quote state.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Same-origin bff - Operations, errors and deployment

Current implementation

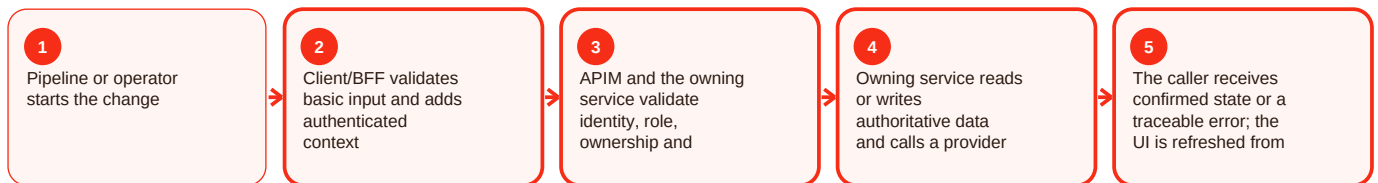
Summary

Same-origin bff is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For same-origin BFF, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed. Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Phone otp - Overview and responsibility

Current implementation

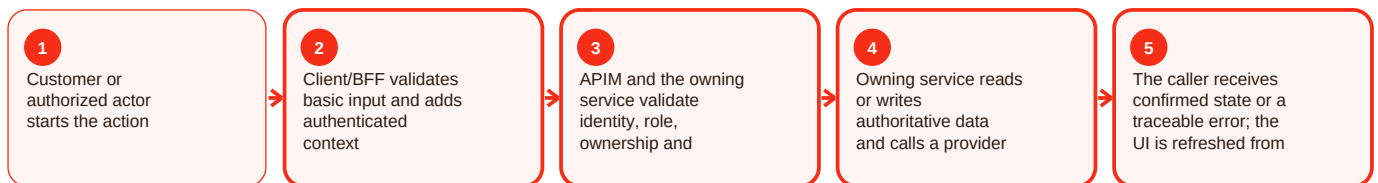
Summary

Phone otp is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For phone OTP, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Phone otp - Functional behavior and data

Current implementation

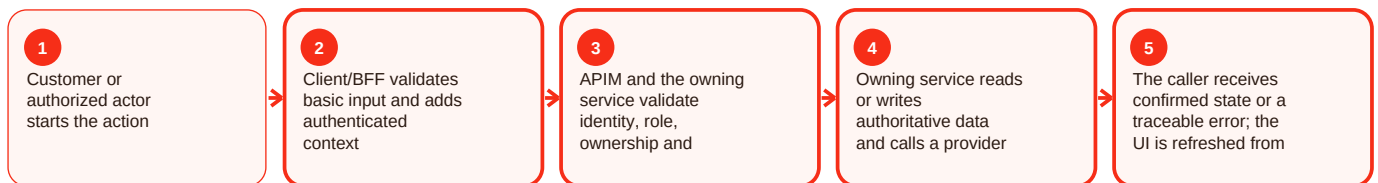
Summary

Phone otp is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For phone OTP, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Phone otp - Operations, errors and deployment

Current implementation

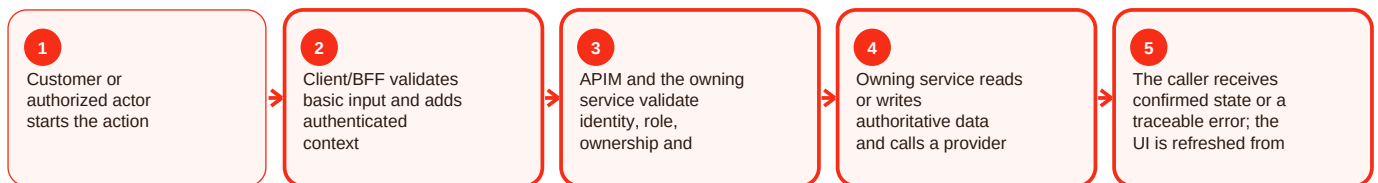
Summary

Phone otp is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For phone OTP, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Secure session cookies - Overview and responsibility

Current implementation

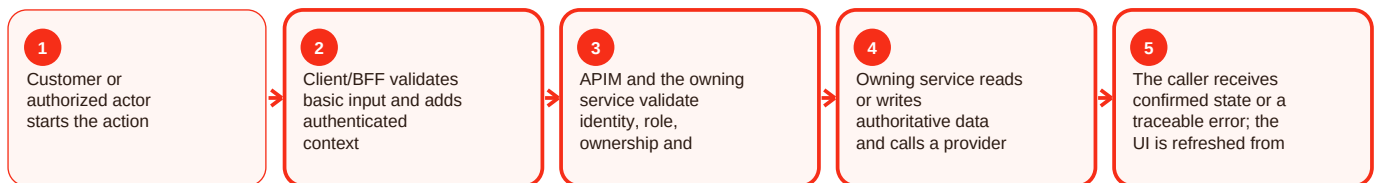
Summary

Secure session cookies is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For secure session cookies, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Secure session cookies - Functional behavior and data

Current implementation

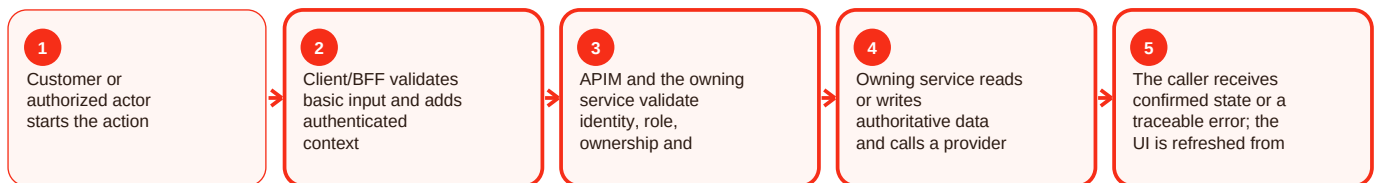
Summary

Secure session cookies is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For secure session cookies, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Secure session cookies - Operations, errors and deployment

Current implementation

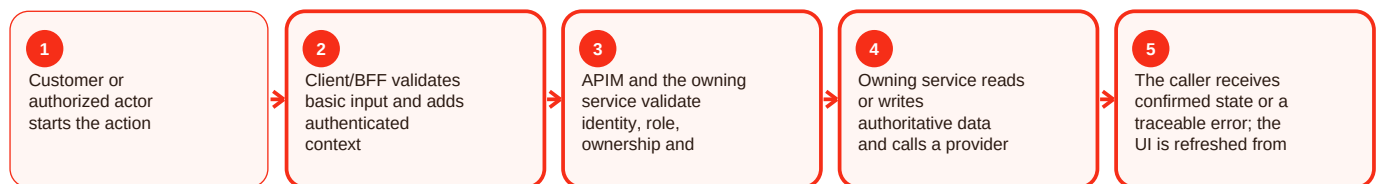
Summary

Secure session cookies is part of the auth area of Craves. Authentication proves who the caller is; authorization decides what that caller may do. Craves uses phone OTP for the current web and JWT-based authorization in backend services, with Entra External ID/PKCE documented for the mobile milestone. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Secure HTTP-only cookies keep web token state away from ordinary browser JavaScript. Backend services evaluate roles such as USER, CHEF and ADMIN and may also enforce object ownership. Mobile milestone guidance stores refresh credentials in Keychain/Keystore and access tokens in memory. For secure session cookies, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Sign-in fails: OTP/session/token is invalid, expired or misconfigured. Resolution: Use the supported sign-in or refresh path and check issuer/audience/configuration.

403 after sign-in: Caller lacks the required role or ownership. Resolution: Confirm intended access; do not bypass authorization.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; services/user-chef-service/README.md; services/order-service/README.md; docs/handover/2026-07-30-customer-mobile-auth-foundation.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Home / discovery - Overview and responsibility

Current implementation

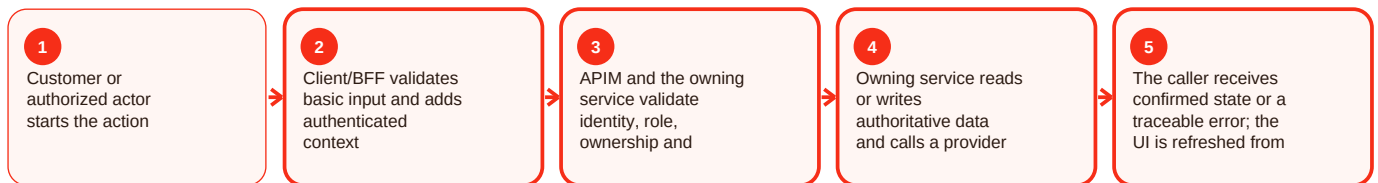
Summary

Home / discovery is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For home/discovery, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Home / discovery - Functional behavior and data

Current implementation

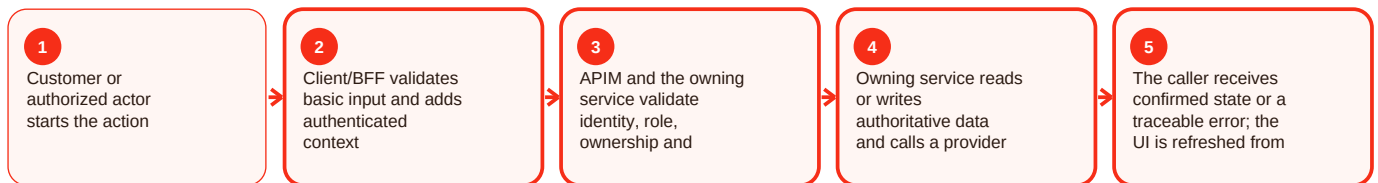
Summary

Home / discovery is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For home/discovery, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Home / discovery - Operations, errors and deployment

Current implementation

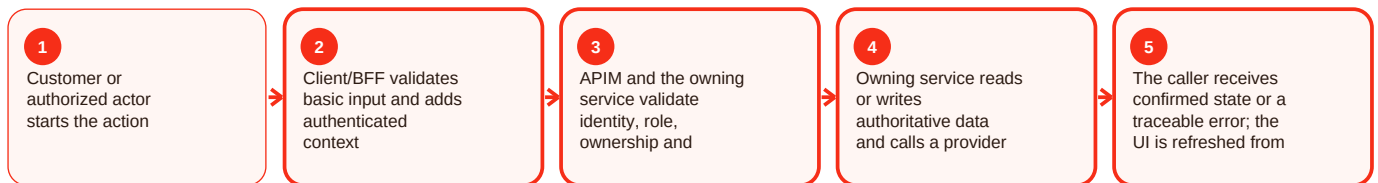
Summary

Home / discovery is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For home/discovery, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Search and meal details - Overview and responsibility

Current implementation

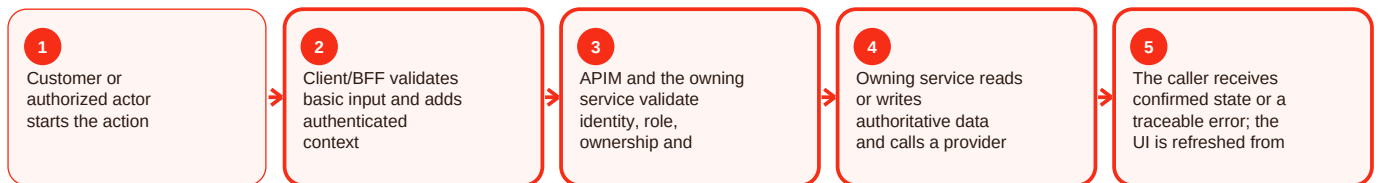
Summary

Search and meal details is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For search and meal details, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Search and meal details - Functional behavior and data

Current implementation

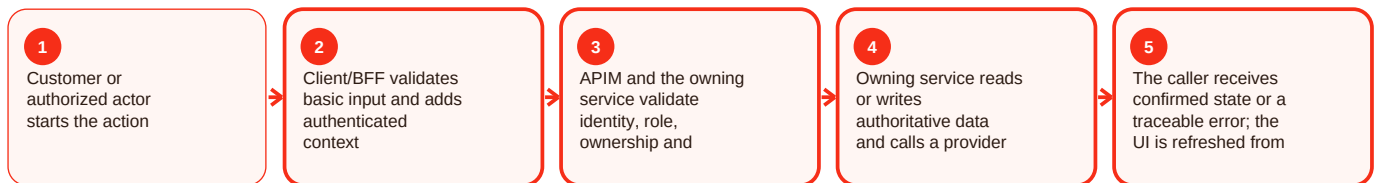
Summary

Search and meal details is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For search and meal details, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius. Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution: Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Search and meal details - Operations, errors and deployment

Current implementation

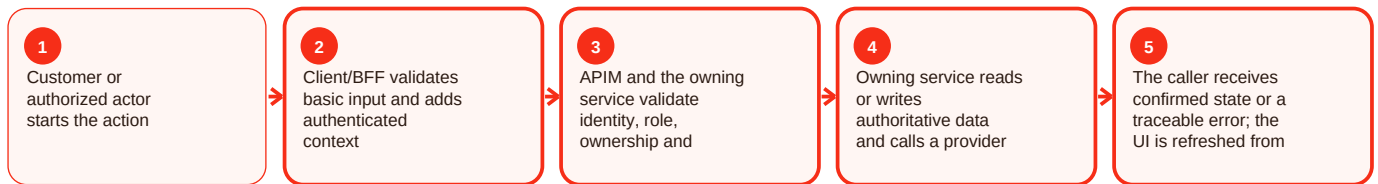
Summary

Search and meal details is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For search and meal details, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius.

Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution:

Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence.

Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cart and quantity - Overview and responsibility

Current implementation

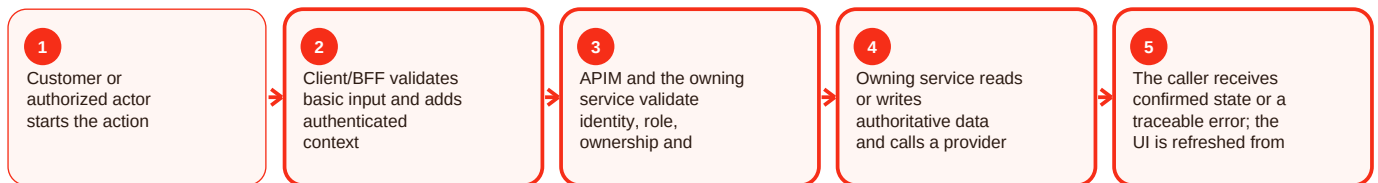
Summary

Cart and quantity is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For cart and quantity, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Checkout rejected: Saved address, cart or catalog state is not valid. Resolution: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. Resolution: Show the backend quote; do not preserve a client-calculated amount.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cart and quantity - Functional behavior and data

Current implementation

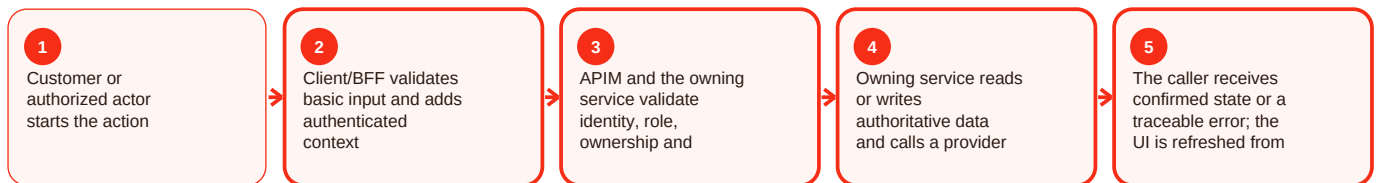
Summary

Cart and quantity is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For cart and quantity, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Checkout rejected: Saved address, cart or catalog state is not valid. Resolution: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. Resolution: Show the backend quote; do not preserve a client-calculated amount.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cart and quantity - Operations, errors and deployment

Current implementation

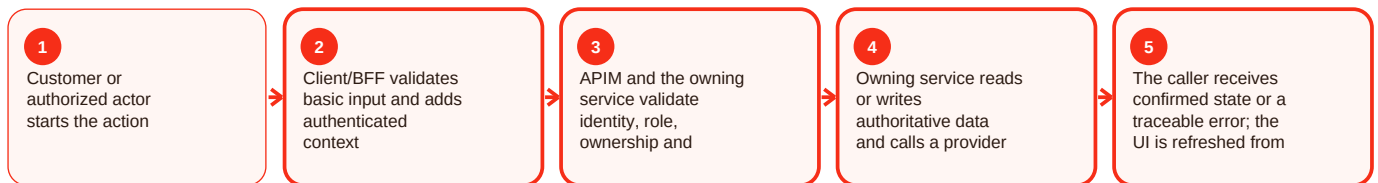
Summary

Cart and quantity is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For cart and quantity, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Checkout rejected: Saved address, cart or catalog state is not valid. Resolution: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. Resolution: Show the backend quote; do not preserve a client-calculated amount.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Checkout quote - Overview and responsibility

Current implementation

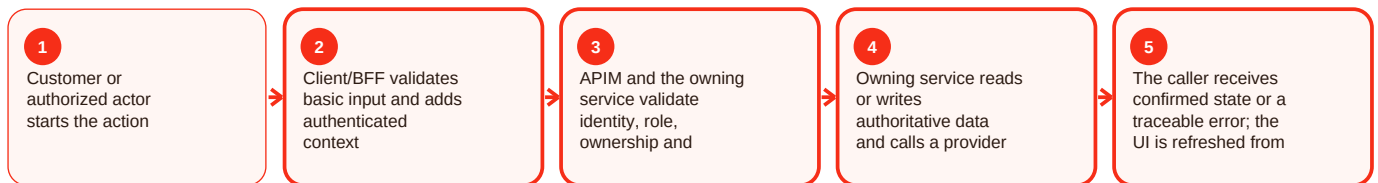
Summary

Checkout quote is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For checkout quote, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Checkout rejected: Saved address, cart or catalog state is not valid. Resolution: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. Resolution: Show the backend quote; do not preserve a client-calculated amount.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Checkout quote - Functional behavior and data

Current implementation

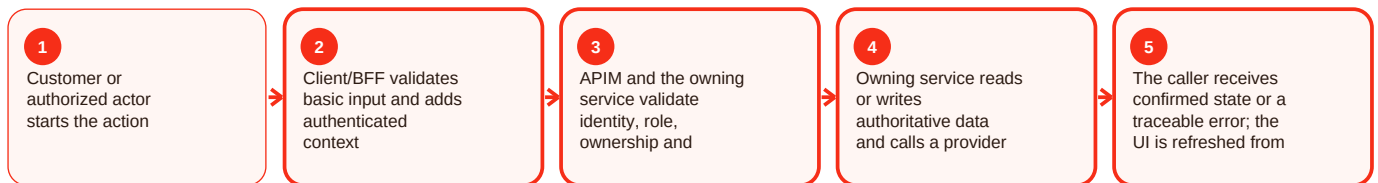
Summary

Checkout quote is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For checkout quote, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Checkout rejected: Saved address, cart or catalog state is not valid. Resolution: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. Resolution: Show the backend quote; do not preserve a client-calculated amount.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Checkout quote - Operations, errors and deployment

Current implementation

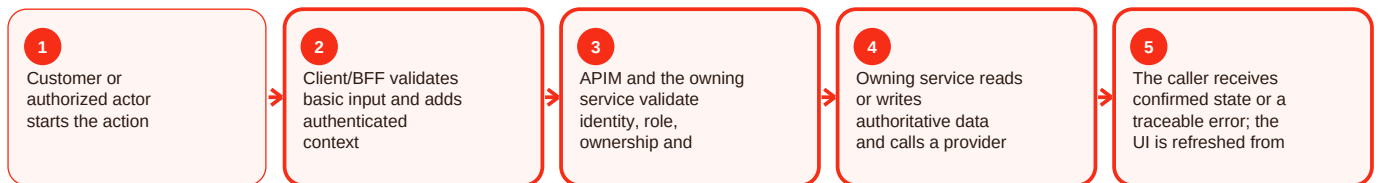
Summary

Checkout quote is part of the checkout area of Craves. Checkout turns a basket into a validated purchase attempt and combines address, authoritative quote, order creation and hosted payment. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser does not decide the payable total. BFF/API calls obtain current backend state; Cashfree payment_session_id is used for hosted payment entry, while backend order/payment paths also document Razorpay acceptance and signed callbacks. For checkout quote, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Checkout rejected: Saved address, cart or catalog state is not valid. Resolution: Reload authoritative cart/address/catalog state and correct the failing condition.

Displayed total changed: Backend revalidated price or availability. Resolution: Show the backend quote; do not preserve a client-calculated amount.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/components/cart/; apps/customer-web-next/src/components/checkout/; services/order-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Address / location - Overview and responsibility

Current implementation

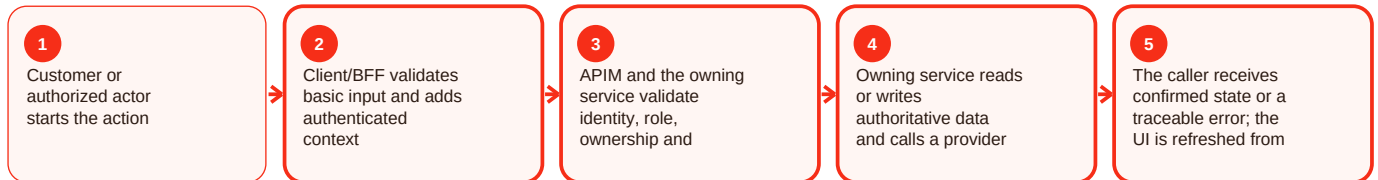
Summary

Address / location is part of the location area of Craves. Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation. For address/location, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Current location unavailable: Permission/device/geocoder failed. Resolution: Allow manual saved-address entry and keep checkout dependent on a valid saved address.

Address rejected at checkout: Address is missing ownership/required fields/coordinates. Resolution: Fix the saved address in User-Chef before placing the order.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Address / location - Functional behavior and data

Current implementation

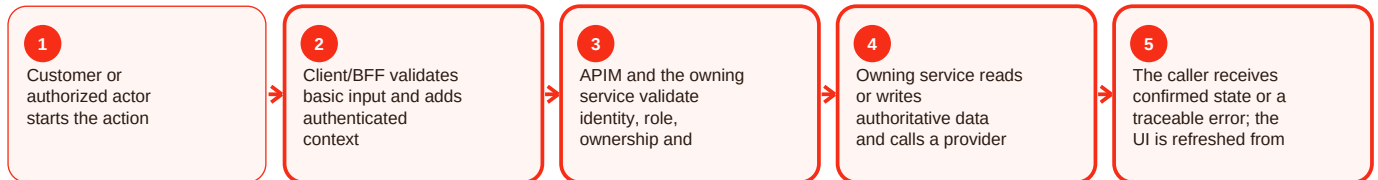
Summary

Address / location is part of the location area of Craves. Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation. For address/location, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Current location unavailable: Permission/device/geocoder failed. Resolution: Allow manual saved-address entry and keep checkout dependent on a valid saved address.

Address rejected at checkout: Address is missing ownership/required fields/coordinates. Resolution: Fix the saved address in User-Chef before placing the order.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Address / location - Operations, errors and deployment

Current implementation

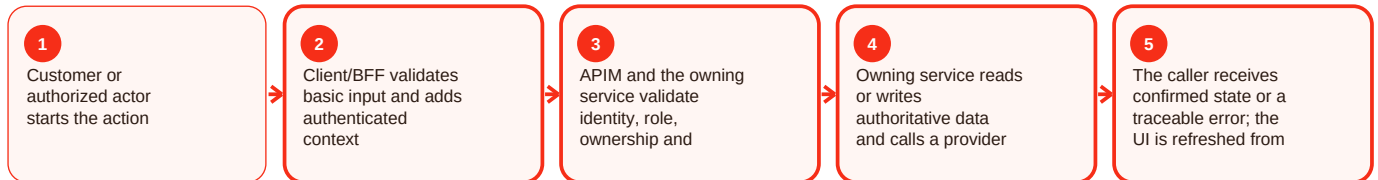
Summary

Address / location is part of the location area of Craves. Location/address features help a customer provide a deliverable address while retaining manual fallback when device permission or geocoding fails. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web contains location/maps/address-dialog modules. Maps are an external dependency; UI convenience does not replace backend serviceability and address validation. For address/location, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Current location unavailable: Permission/device/geocoder failed. Resolution: Allow manual saved-address entry and keep checkout dependent on a valid saved address.

Address rejected at checkout: Address is missing ownership/required fields/coordinates. Resolution: Fix the saved address in User-Chef before placing the order.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/src/components/location/; apps/customer-web-next/src/components/maps/; apps/customer-web-next/src/lib/location/; docs/handover/2026-08-06-customer-chef-precise-ui-address-dialog.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cashfree payment - Overview and responsibility

Current implementation

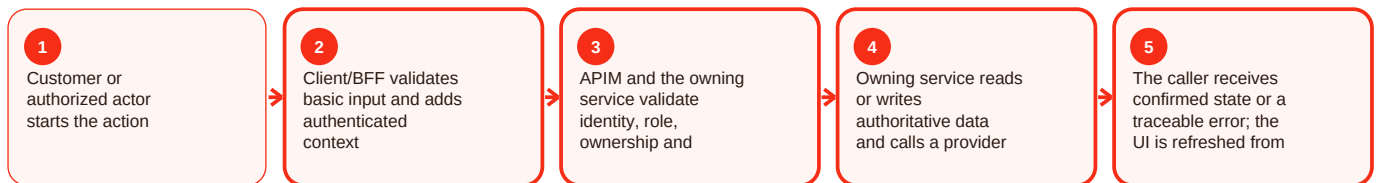
Summary

Cashfree payment is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Cashfree payment, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cashfree payment - Functional behavior and data

Current implementation

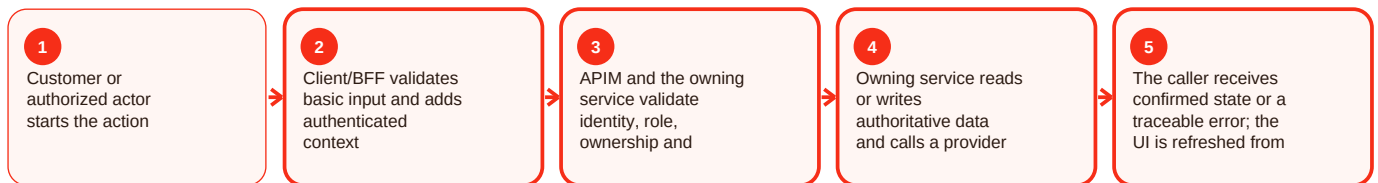
Summary

Cashfree payment is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Cashfree payment, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Cashfree payment - Operations, errors and deployment

Current implementation

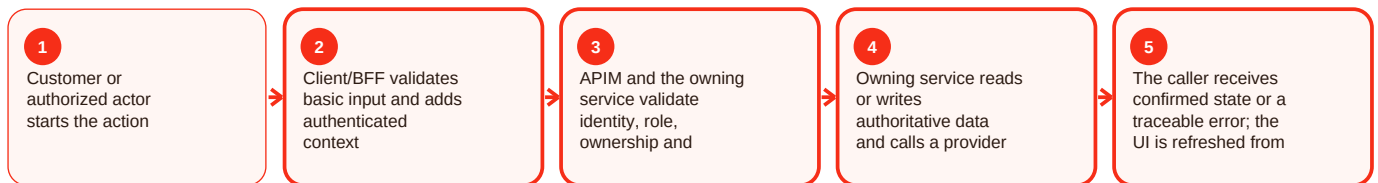
Summary

Cashfree payment is part of the payment area of Craves. Payments are separated from raw card handling so the Craves UI can use provider-hosted entry while the backend protects order/payment state. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The web documents Cashfree hosted payment sessions. order-service verifies Razorpay callbacks with HMAC SHA-256 over the raw body using X-Razorpay-Signature, and integration-service documents normalized payment-link operations. For Cashfree payment, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Payment result is unclear: Provider status/callback is delayed or ambiguous. Resolution: Verify through the backend/provider reconciliation path before retrying.

Webhook rejected: Signature validation failed. Resolution: Verify the raw body with the server secret; never disable signature checks.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/lib/cashfree/; services/order-service/README.md; services/integration-service/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Confirmation / history / tracking - Overview and responsibility

Current implementation

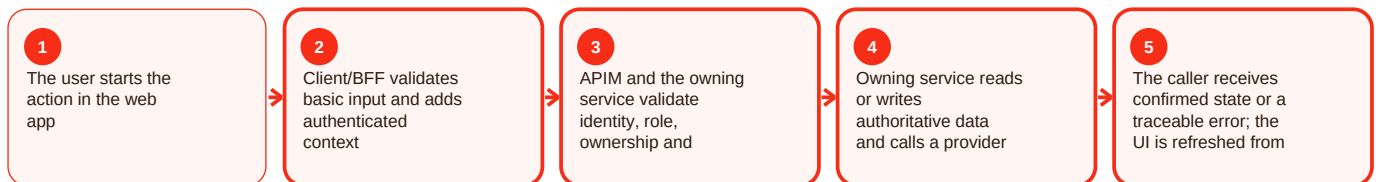
Summary

Confirmation / history / tracking is part of the frontend area of Craves. The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state. For confirmation/history/tracking, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Generic upstream error: BFF rejected malformed/unexpected backend response. Resolution: Use request ID and compare the current response contract.

Amount differs: Client state is stale. Resolution: Render backend-authoritative cart/quote state.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app; apps/customer-web-next/src/components; apps/customer-web-next/src/features/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Confirmation / history / tracking - Functional behavior and data

Current implementation

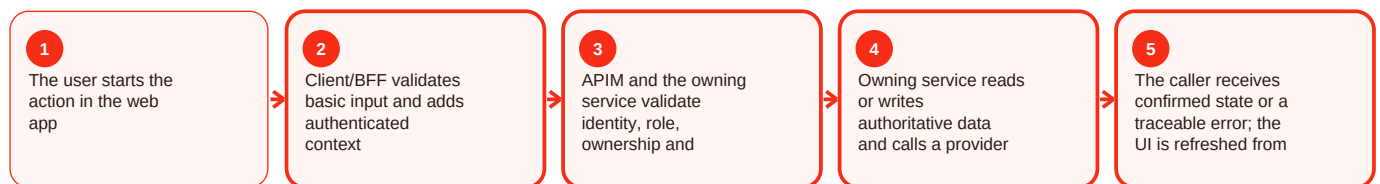
Summary

Confirmation / history / tracking is part of the frontend area of Craves. The primary customer-facing web implementation is apps/customer-web-next, a Next.js 14 App Router application containing customer, chef and newer admin/support modules. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The browser uses same-origin BFF handlers, secure HTTP-only token cookies and APIM at /api/v1. Firebase phone OTP is used for the current web sign-in flow, and backend responses remain authoritative for price, availability and order state. For confirmation/history/tracking, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Generic upstream error: BFF rejected malformed/unexpected backend response. Resolution: Use request ID and compare the current response contract.

Amount differs: Client state is stale. Resolution: Render backend-authoritative cart/quote state.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; apps/customer-web-next/src/app/; apps/customer-web-next/src/components/; apps/customer-web-next/src/features/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Confirmation / history / tracking - Operations, errors and deployment

Current implementation

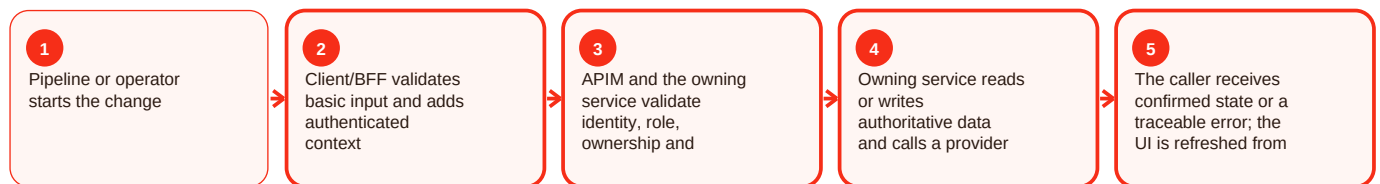
Summary

Confirmation / history / tracking is part of the devops area of Craves. The repository has broad CI/CD references across backend, frontend, mobile, APIM, data, privacy, security, smoke and release controls. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

customer-web-next README describes ACR build/push, Container App revision deployment, readiness checks and restoration of the prior image/revision on failed readiness. Pipeline names not resolvable at their cited path are classified as reference-only instead of assigned invented triggers. For confirmation/history/tracking, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Pipeline succeeds but app is not ready: Revision is still starting or readiness was not awaited. Resolution: Check the actual revision and health/readiness after deployment.

New revision is unhealthy: Image, configuration, migration or dependency failed.

Resolution: Rollback to the known-good image/revision and verify recovery.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/customer-web-next/README.md; repository YAML search; docs/handover/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Plans / subscriptions - Overview and responsibility

Current implementation

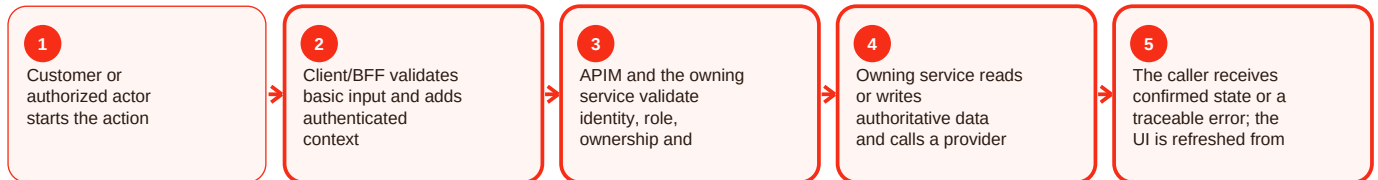
Summary

Plans / subscriptions is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For plans/subscriptions, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Lifecycle action rejected: Current subscription state does not allow it. Resolution: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. Resolution: Document the decision before implementing it.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/subscription-service/README.md; apps/customer-web-next/src/components/plans/; apps/customer-web-next/src/components/subscriptions/; commit 3225f7fa. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Plans / subscriptions - Functional behavior and data

Current implementation

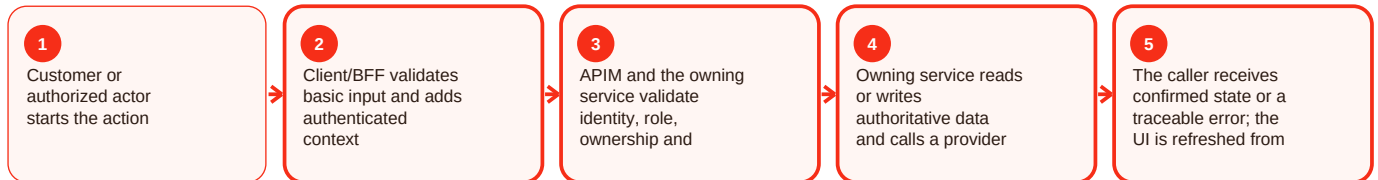
Summary

Plans / subscriptions is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For plans/subscriptions, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Lifecycle action rejected: Current subscription state does not allow it. Resolution: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. Resolution: Document the decision before implementing it.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/subscription-service/README.md; apps/customer-web-next/src/components/plans/; apps/customer-web-next/src/components/subscriptions/; commit 3225f7fa. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Plans / subscriptions - Operations, errors and deployment

Current implementation

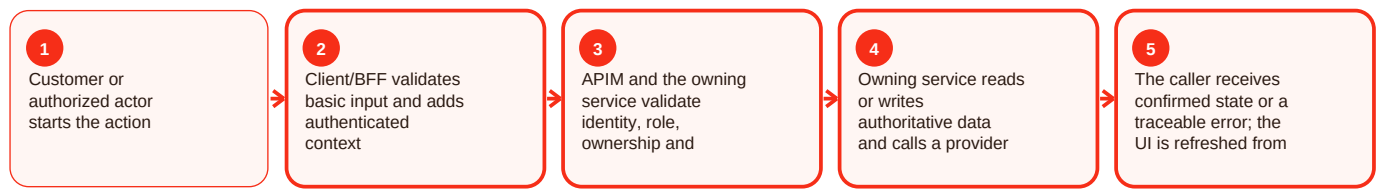
Summary

Plans / subscriptions is part of the subscription area of Craves. Subscriptions support repeat service and entitlements on top of one-off ordering. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

subscription-service migrations cover plans, subscriptions, payment identifiers/tokens, idempotency, request fingerprints and usage counters. The latest observed main commit adds shared entitlements and gated services. For plans/subscriptions, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Lifecycle action rejected: Current subscription state does not allow it. Resolution: Reload current state and use a supported transition.

Commercial rule missing: Product/Finance decision is intentionally not defined. Resolution: Document the decision before implementing it.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/subscription-service/README.md; apps/customer-web-next/src/components/plans/; apps/customer-web-next/src/components/subscriptions/; commit 3225f7fa. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef mode - Overview and responsibility

Current implementation

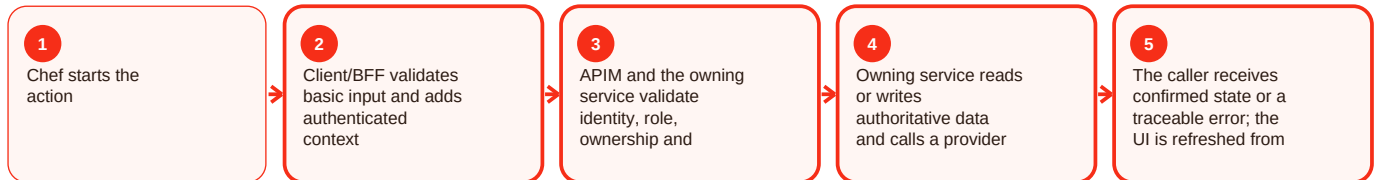
Summary

Chef mode is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef mode, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef mode - Functional behavior and data

Current implementation

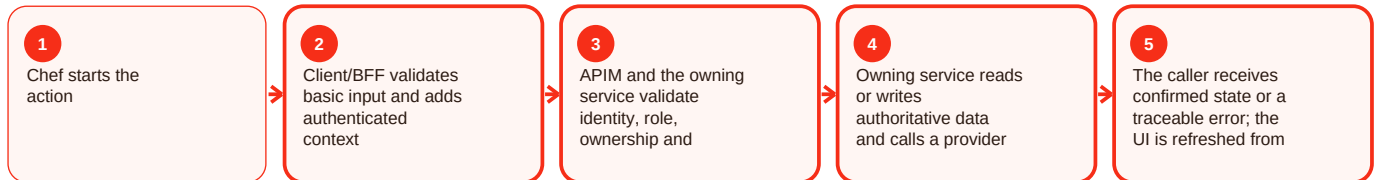
Summary

Chef mode is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef mode, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef mode - Operations, errors and deployment

Current implementation

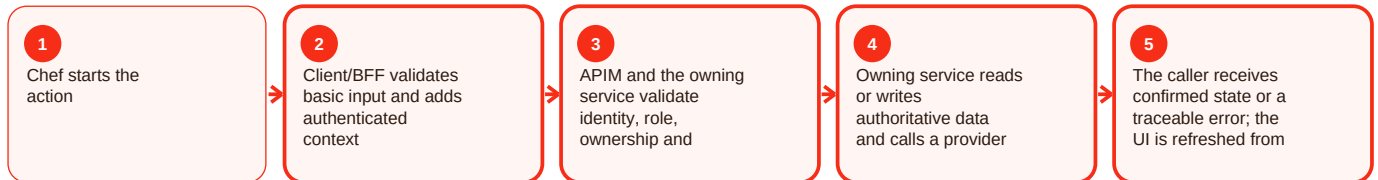
Summary

Chef mode is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef mode, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef onboarding - Overview and responsibility

Current implementation

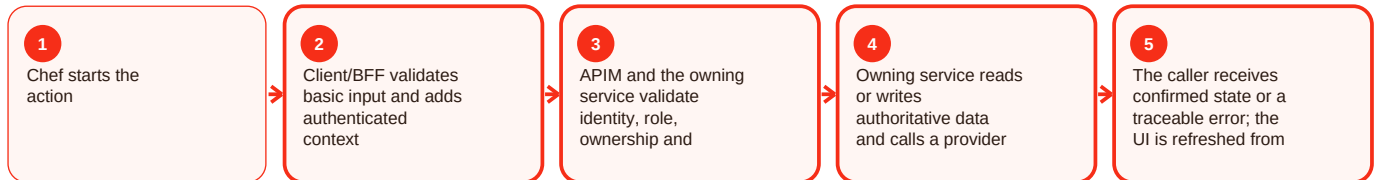
Summary

Chef onboarding is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef onboarding, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef onboarding - Functional behavior and data

Current implementation

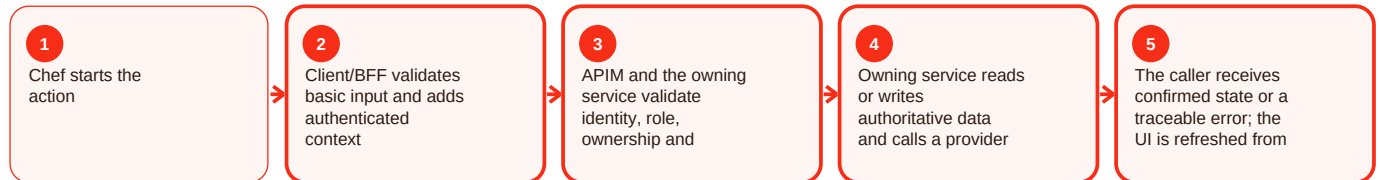
Summary

Chef onboarding is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef onboarding, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef onboarding - Operations, errors and deployment

Current implementation

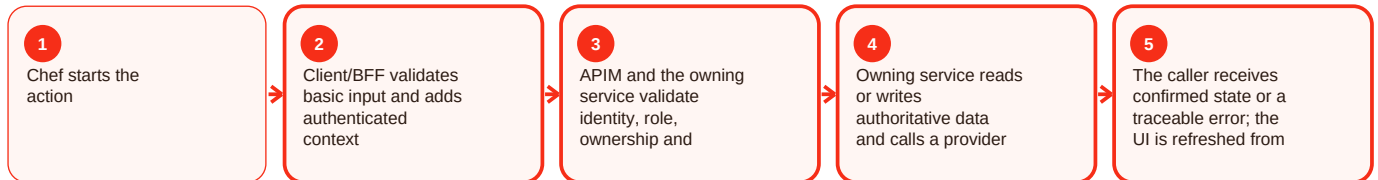
Summary

Chef onboarding is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef onboarding, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef menu / kitchen / orders - Overview and responsibility

Current implementation

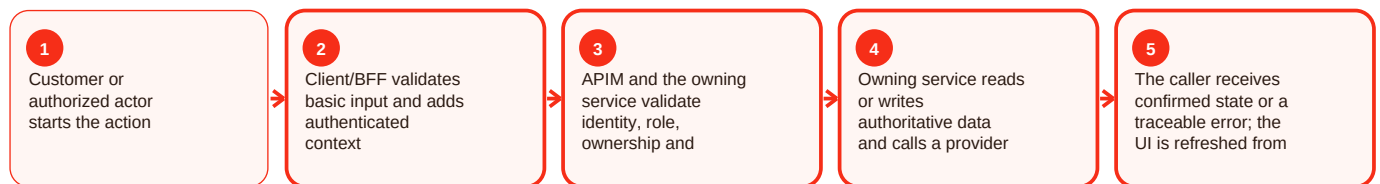
Summary

Chef menu / kitchen / orders is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For chef menu/kitchen/orders, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius.

Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution:

Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence.

Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef menu / kitchen / orders - Functional behavior and data

Current implementation

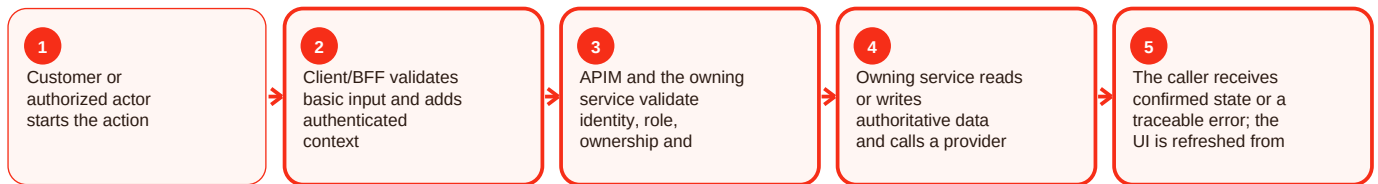
Summary

Chef menu / kitchen / orders is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For chef menu/kitchen/orders, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius.

Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution:

Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence.

Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef menu / kitchen / orders - Operations, errors and deployment

Current implementation

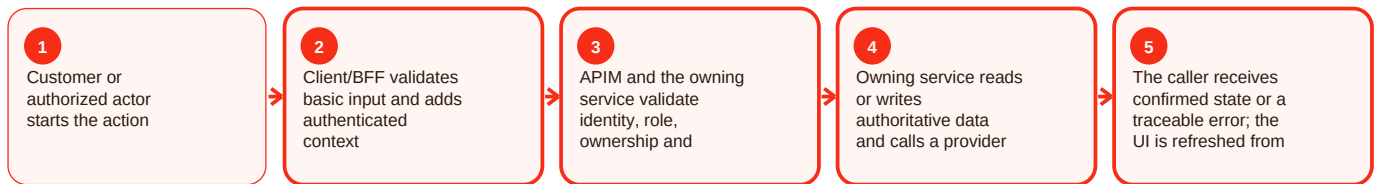
Summary

Chef menu / kitchen / orders is part of the catalog area of Craves. Catalog is the source of browseable meal and cuisine information used by discovery and commerce validation. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

catalog-service is a Spring Boot/Flyway service with JWT controls. The web contains discovery, meals, search and selector modules. Administrative catalog operations are described by APIM guidance, including bulk pause/import and SHA-256 integrity checks. For chef menu/kitchen/orders, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Nearby results empty: No active geocoded kitchen with sellable items in query radius.

Resolution: Confirm kitchen coordinates, active menu items and caller radius.

Menu item cannot sell: Packaging/availability/status metadata incomplete. Resolution:

Chef must provide valid required metadata and make the item available.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/catalog-service/README.md; apps/customer-web-next/src/components/discovery/; apps/customer-web-next/src/components/meals/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence.

Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef earnings - Overview and responsibility

Current implementation

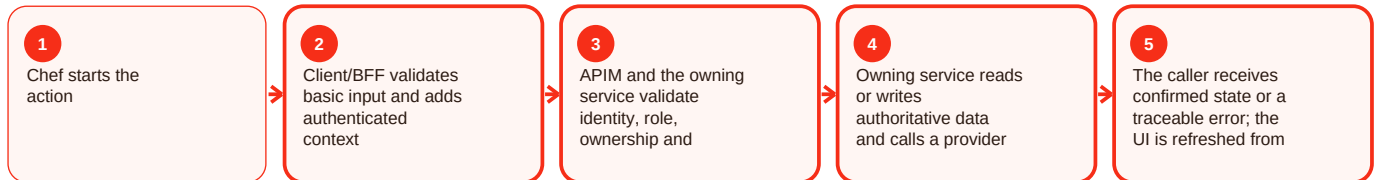
Summary

Chef earnings is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef earnings, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef earnings - Functional behavior and data

Current implementation

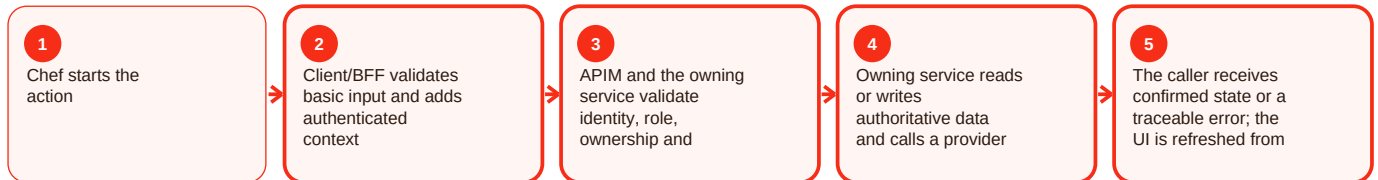
Summary

Chef earnings is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef earnings, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Chef earnings - Operations, errors and deployment

Current implementation

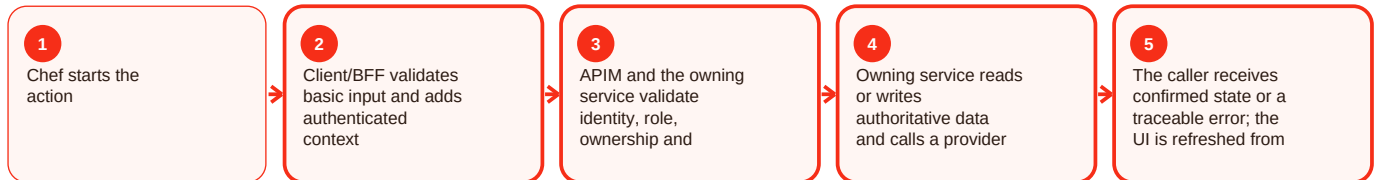
Summary

Chef earnings is part of the chef area of Craves. Chef mode is the supply side of the marketplace: onboarding, verification, menus, kitchen/order operations and earnings. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

user-chef-service persists chef profiles and verification state. The frontend can switch context, but the server remains the authority: showing Chef mode never grants a CHEF role by itself. For chef earnings, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: services/user-chef-service/README.md; apps/customer-web-next/src/components/chef/; apps/customer-web-next/src/components/chef-mode/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin / support modules - Overview and responsibility

Current implementation

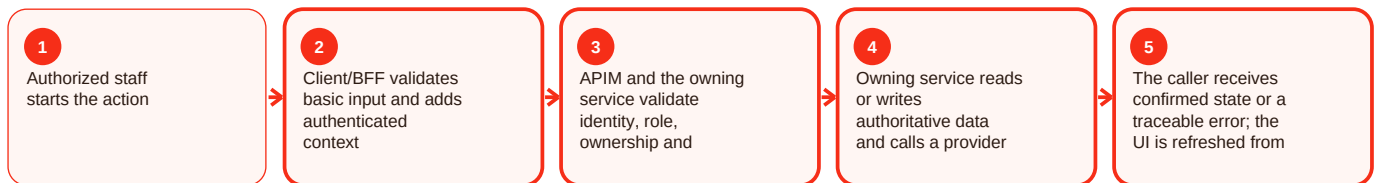
Summary

Admin / support modules is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin/support modules, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin / support modules - Functional behavior and data

Current implementation

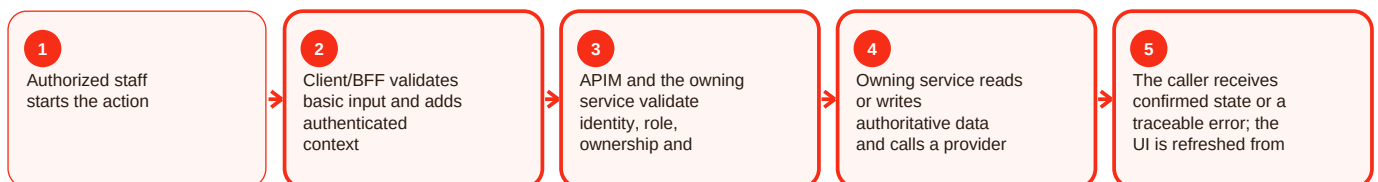
Summary

Admin / support modules is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin/support modules, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Admin / support modules - Operations, errors and deployment

Current implementation

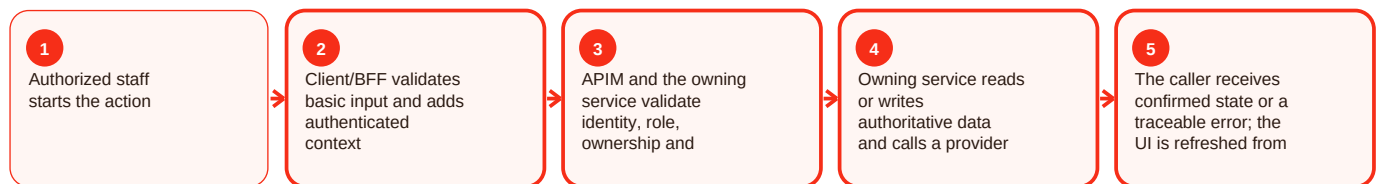
Summary

Admin / support modules is part of the admin area of Craves. Administrative tooling lets authorized staff review marketplace state unavailable to ordinary customers and chefs. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

The repository contains legacy apps/admin plus newer customer-web-next admin/support/chef-review modules. APIM guidance places admin operations in a subscription-protected product with additional role checks. For admin/support modules, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

Admin action denied: Caller lacks admin role/product permission. Resolution: Use approved staff access process and least privilege.

Operation changes unexpected data: Wrong record/state selected. Resolution: Stop, capture audit/request evidence and follow the documented recovery path.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: apps/admin/README.md; apps/customer-web-next/src/features/admin/; apps/customer-web-next/src/features/chef-review/; infra/apim/README.md. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Consent / data rights - Overview and responsibility

Current implementation

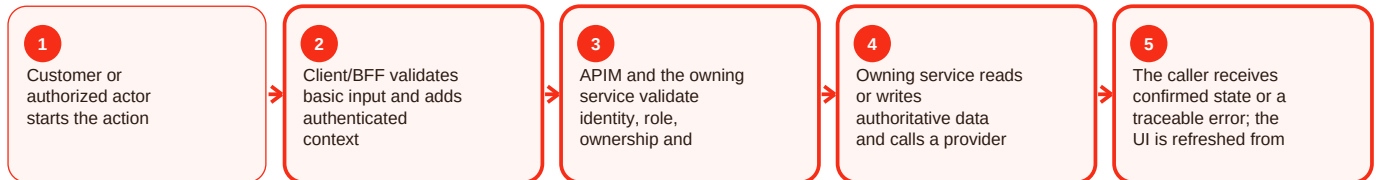
Summary

Consent / data rights is part of the privacy area of Craves. Privacy capability includes consent preferences plus recent customer data export/deletion implementation on main. In practical terms, this capability should make one responsibility clear: who starts the action, which component validates it, which service owns the final state, and what the user sees when the action succeeds or fails.

Technical explanation

Recent commits record centralized consent enforcement, a consent-center UI/backend wiring, customer export and deletion workflows. Exact legal retention periods are not invented where source is silent. For consent/data rights, the important engineering rule is that the user interface requests or displays state; the owning backend or gateway policy decides whether the request is valid. Data, identity, provider calls and deployment configuration remain inside their documented boundaries, which prevents a convenient screen or integration from becoming a second source of truth.

Process flow



Common issues and resolution

User sees stale state: Client cache/local state differs from backend. Resolution: Reload from the owning service.

Dependency failure: Provider or downstream is unavailable. Resolution: Preserve domain consistency and retry only when safe.

Validation and evidence

Direct repository evidence supports this capability or architecture boundary. Evidence: commit 188385e; commit 93ebfa4; commit 5dd7971; apps/customer-web-next/. Validate using authoritative state, health/readiness where relevant, and request/correlation evidence. Never paste credentials, OTPs, tokens or provider secrets into tickets or documentation.

Operational checklist and documented gaps

Before making a change

- Confirm the intended actor and environment.
- Confirm the current authoritative state before changing anything.
- Capture HTTP status, stable error code and request/correlation ID without secrets.
- Validate the owning component health and required dependency/configuration.
- After the action, reload the state from the backend and confirm the expected result.

Repository gaps that must stay visible

- APIM README cites `infra/apim/craves-openapi.yaml`, but that literal artifact was not resolvable on main during this evidence snapshot.
- `customer-web-next` README names `azure-pipelines-customer-web-next.yml`, but that literal root path was not resolvable; deployment behavior described by the README is retained while trigger/variable details are not invented.
- Native mobile has strong milestone identity/API/CI evidence, but a clearly complete runnable React Native application was not established on the reviewed main snapshot.
- Legacy preview URLs prove historical deployment references rather than current production routing.
- Commercial values such as commissions, payout percentages, subscription pricing, catering thresholds and SLAs are not fabricated where source is silent.

Definition of done

A change is complete only when the intended behavior works, authorization still fails closed, authoritative data is correct, health/readiness are good, monitoring or correlation evidence is available, the rollback path remains valid, and the documentation has been updated if the contract or operating procedure changed.